COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09 - 04 - 26	**DATE SUBMITTED:**	09 - 08 - 26
NAME:	Paula Esguerra	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA NO. _7__ Sort Techniques

1. PROCEDURE OUTPUT

Array of Values for Sort Algorithm Testing

Screenshot

```
int main() {

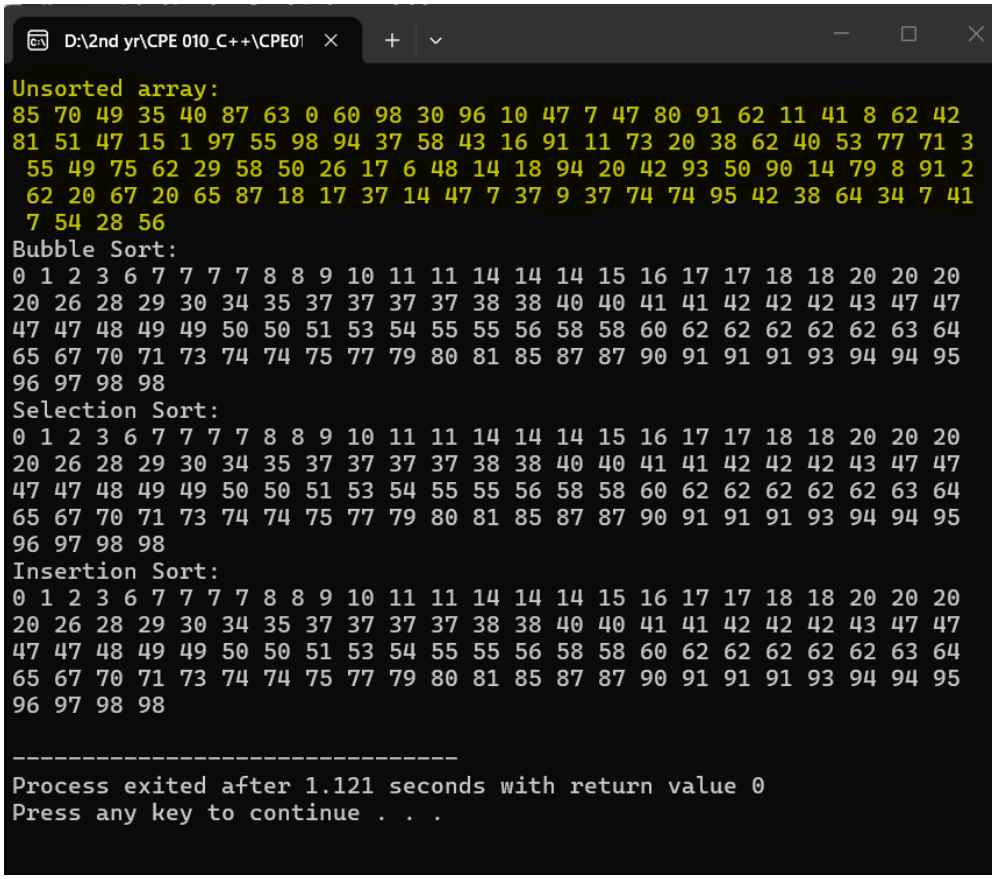
    // Create an array with unsorted list with 100 maximum size
    int unsorted_arr[max_size] = {};

    // Use time (0) as seed for random values
    std::srand(std::time(0));

    // Populate the array
    for (int i = 0; i < max_size; i++) {
        unsorted_arr[i] = std::rand() % 100;
    }

    // Display the unsorted array
    std::cout << "Unsorted array: " << std::endl;
    displayArr(unsorted_arr);
```

Output



Observations

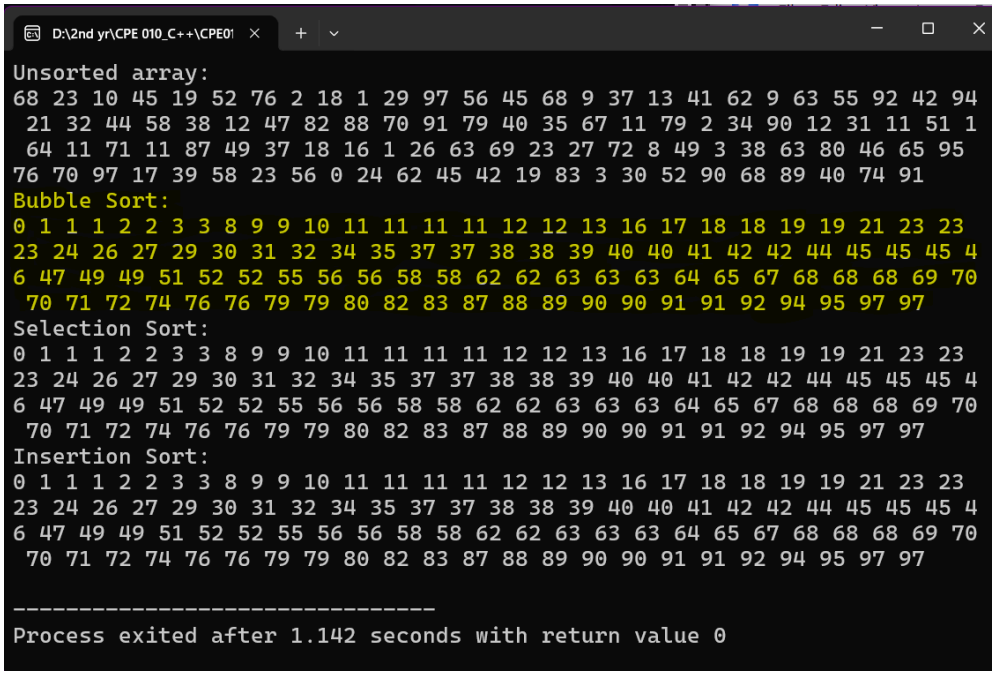
This is the original list, a random list, 100 elements, to be used in the sorting code of algorithms, this is the starting point and print to see, how it will turned out after it will be sorted in different ways.

Bubble Sort Technique	
Code	Header (sortingAlgo.h) main.cpp

```
5   template <typename T>
6   void bubbleSort(T arr[], size_t arrSize) {
7       // Step 1: for i = 0 to N-1 repeat inner loop
8       for (int i = 0; i < arrSize - 1; i++) {
9
10          // Step 2: Inner loop that for K = i+1 to N-1 repeat
11          for (int j = i+1; j < arrSize; j++) {
12
13              // Step 3: Swap if arr[j] > arr[i]
14              if (arr[i] > arr[j]) {
15                  // Swap arr[j] and arr[i]
16                  std::swap(arr[j], arr[i]);
17              }
18          }
19      }
20  }
21
```

```
27
28      // Sort the array using bubble sort
29      std::cout << "Bubble Sort: " << std::endl;
30      bubbleSort(unsorted_arr, max_size);
31      displayArr(unsorted_arr);
32
```

Output

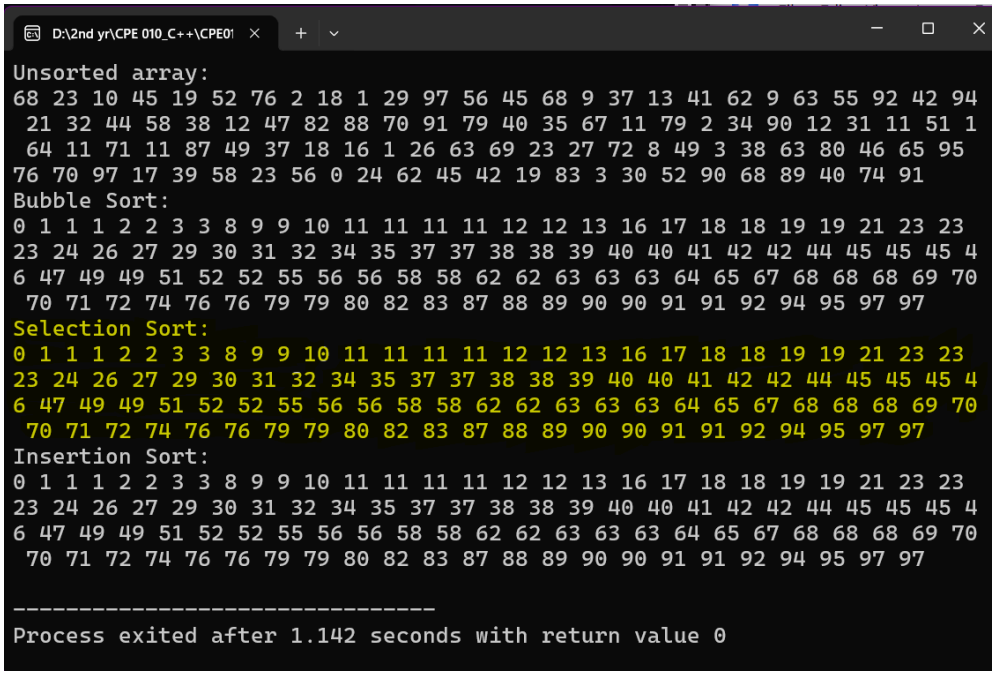


Observations

The bubblesort arranges the list from smallest to the largest, starting at the very first position, then it will compare the next node, then compare the next one until it satisfy the condition, the condition is when it meets the node bigger, it will stop and choose the spot. Then, the next node will keep on moving spot by spot until the entire group is perfectly ordered. This behaves like the selection sort, from one position from left to right.

Selection Sort Algorithm	

Code	Header (sortingAlgo.h)
	`template <typename T>`
	`// Array, current position, size of array`
	`int routineSmallest(T arr[], int K, const int arrSize) {`
	`    int position = K;`
	`    T smallestElem = arr[K];`
	`    for(int j = K+1; j < arrSize; j++){`
	`        if(arr[j] < smallestElem){`
	`            smallestElem = arr[j];`
	`            position = j;`
	`        }`
	`    }`
	`    return position;`
	`}`
	`template<typename T>`
	`void selectionSort(T arr[], size_t arrSize){`
	`    int POS, temp, pass;`
	`    for(int i=0; i < arrSize; i++){`
	`        POS = routineSmallest(arr, i, arrSize);`
	`        // Swapping:`
	`        temp = arr[i];`
	`        arr[i] = arr[POS];`
	`        arr[POS] = temp;`
	`        // For verification and counting of number of passes that was done`
	`        pass++;`
	`    }`
	`}`
	Main.cpp
	`// Sort the array using selection sort`
	`std::cout << "Selection Sort: " << std::endl;`
	`selectionSort(unsorted_arr, max_size);`
	`displayArr(unsorted_arr);`

Output	
Observations	In the bubblesort, numbers are swapped places one by one, selection sort on the other hand, uses routineSmallest this is liek an assist where it makes the sorting less chaotic. It starts are on the first node, then instead od swapping constantly, it just continues until it find the one that fulfill the condition, it doesn't move anyone, just store on the memory where it need to move, then it switches directly to the position, then repeats the process.

Insertion Sort Algorithm

Code

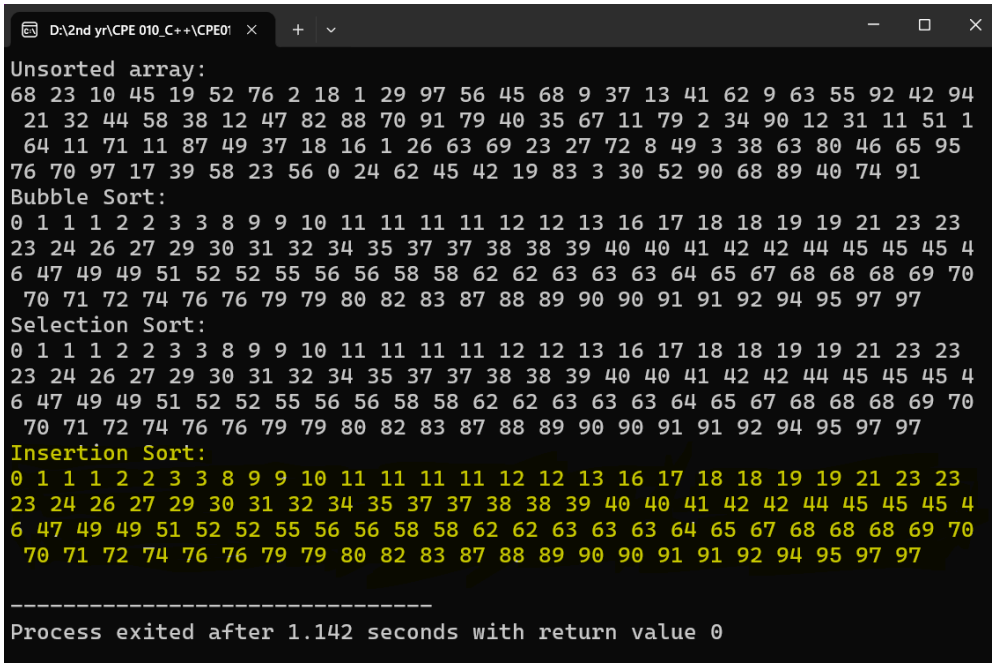
Header(sortingAlgo.h)

```
53
54  template<typename T>
55  void insertionSort(T arr[], size_t arrSize){
56      int K = 1, J;
57      T temp;
58
59      while(K < arrSize){
60          temp = arr[K];
61          J = K - 1;
62
63          while(J >= 0 && temp < arr[J]){
64              arr[J+1] = arr[J];
65              J--;
66          }
67
68          arr[J+1] = temp;
69          K++;
70      }
71  }
```

main.cpp

```
// Sort the array using insertion sort
std::cout << "Insertion Sort: " << std::endl;
insertionSort(unsorted_arr, max_size);
displayArr(unsorted_arr);
```

Output



Observations

Insertion sort is like the insertion when playing cards, instead of scanning the whole list, it builds another list and arrange them one by one. It doesn't move the first node but to the second, put on to the second list, then look to the left, is this left is bigger than the one on the second list, it places the one on the second list to the right, up until the left finds the condition it will fulfill, it repeat the same process until the whole list is arranged.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B:	
Code	Header (sortingAlgo.h) main.cpp

Header (sortingAlgo.h)

```
73  template<typename T>
74  void insertionSort_two(T arr[], size_t arrSize){
75      for (int i = 1; i <arrSize; i++){
76          int temp = arr[i];
77          int j = i - 1;
78
79          while(j >= 0 && arr[j] > temp){
80              arr[j + 1] = arr[j];
81              j--;
82          }
83          arr[j+1] = temp;
84      }
85  }
86
```

main.cpp

```
1   #include <iostream>
2   #include <cstdlib>
3   #include <ctime>
4   #include <string>
5   #include "sortingAlgo.h"
6
7   const int max_size =101;
8
9   void displayArr(int arr[], int size);
10  void winnerVotes(int arr[], int size);
11
12
13  int main() {
14
15      // Create an array with unsorted list with 101 maximum size
16      int unsorted_arr[max_size] = {};
17
18      // Use time (0) as seed for random values
19      std::srand(std::time(0));
20
21      // Populate the array
22      for (int i = 0; i < max_size; i++) {
23          unsorted_arr[i] = (std::rand() % 5) + 1;
24      }
25
```

```
25
26      // Display the unsorted array
27      std::cout << "Unsorted array: " << std::endl;
28      displayArr(unsorted_arr, max_size);
29      std::cout << std::endl;
30
31      std::cout << "Sorted Array using Insertion Sort: " << std::endl;
32      insertionSort_two(unsorted_arr, max_size);
33      displayArr(unsorted_arr, max_size);
34      std::cout << std::endl;
35
36      // Sort the array using bubble sort
37      std::cout << "Election Results: " << std::endl;
38      winnerVotes(unsorted_arr, max_size);
39      std::cout << std::endl;
```

```
55
56  void displayArr(int arr[], int size) {
57      for (int i = 0; i < max_size; i++) {
58          std::cout << arr[i] << " ";
59      }
60      std::cout << std::endl;
61  }
62
63  void winnerVotes(int arr[], int size){
64      std::string candidates[5] = {
65          "Bo Dalton Capistrano",
66          "Cornelius Raymon Agustin",
67          "Deja Jayla Banaga",
68          "Lalla Brielle Yabut",
69          "Franklin Relano Castro"
70      };
71
72      int vote_counts[5] = {0,0,0,0,0};
73
74      for (int i = 0; i < size; i++){
75          int vote = arr[i];
76          if(vote >= 1 && vote <= 5){
77              vote_counts[vote - 1]++;
78          }
79      }
80
```

```
81      int max_votes = -1;
82      int winner = 0;
83      bool tie = false;
84
85      for(int i = 0; i <5; i++){
86          std::cout << "Candidate " << (i + 1) << ":" << candidates[i] << " with " <<
87                       vote_counts[i] << " votes" << std::endl;
88
89          if(vote_counts[i] > max_votes){
90              max_votes = vote_counts[i];
91              winner = i;
92              tie = false;
93          }else if(vote_counts[i] == max_votes){
94              tie = true;
95          }
96      }
```

```
98      std::cout << std::endl;
99      if(tie){
100         std::cout << "The election is a TIE!" << std::endl;
101     }else{
102         std::cout << "The winner of this election is " << candidates[winner]
103                   << "!!!!" <<  " with the total votes of " << max_votes << "!"
104                   << std::endl;
105     }
106 }
```

Output

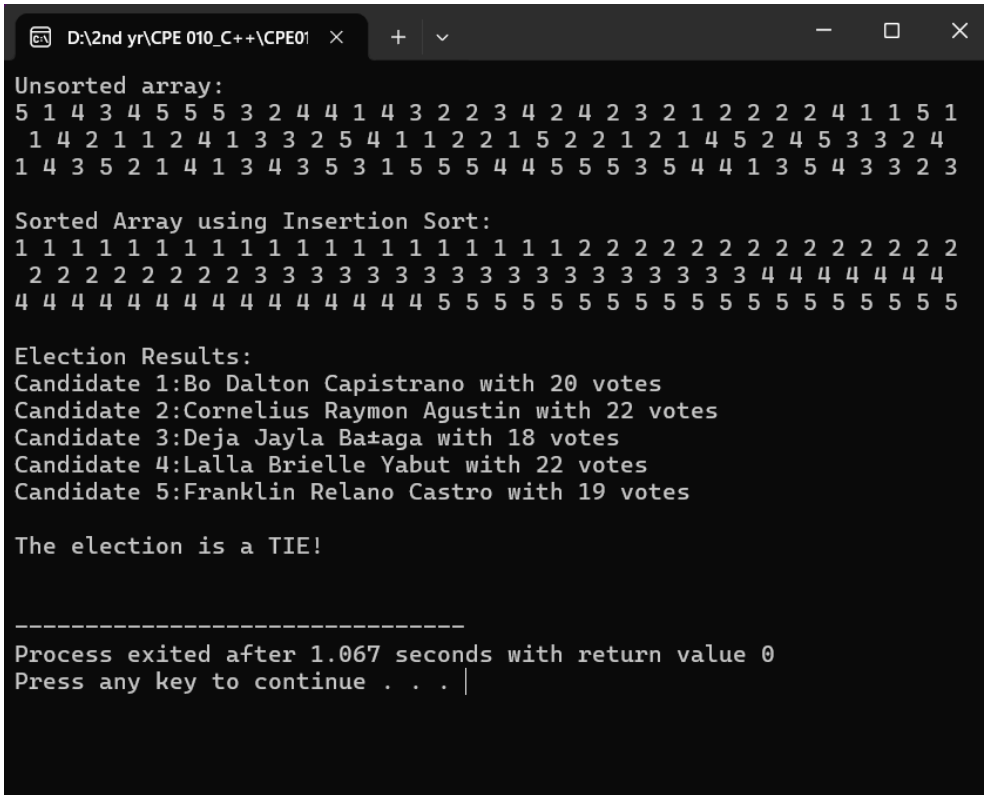
```
D:\2nd yr\CPE 010_C++\CPE01
Unsorted array:
4 1 3 1 3 4 1 5 3 4 1 1 3 2 2 5 2 3 1 3 2 2 1 5 2 1 2 2 2 5 5
5 5 1 3 1 5 3 2 4 4 4 2 1 1 1 2 4 4 2 3 2 3 4 1 4 1 4 1 5 5 3
2 2 3 1 2 5 5 2 2 3 4 3 1 1 3 2 3 1 1 3 3 1 4 3 3 3 4 5 4 3 3
1 4 5 3 3 2 1 2

Sorted Array using Insertion Sort:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5
5 5 5 5 5 5 5 5

Election Results:
Candidate 1:Bo Dalton Capistrano with 24 votes
Candidate 2:Cornelius Raymon Agustin with 22 votes
Candidate 3:Deja Jayla Banaga with 25 votes
Candidate 4:Lalla Brielle Yabut with 16 votes
Candidate 5:Franklin Relano Castro with 14 votes

The winner of this election is Deja Jayla Banaga!!!! with the
total votes of 25!


--------------------------------
Process exited after 1.085 seconds with return value 0
Press any key to continue . . .
```



Observation	I needed to have 100 balots, so I set up the random array to 101 to be indexed to 0 to 100, then I set up the code where the balots are only 1 to 5, since the candidates are only only 1 to 5, so they only need to choose between 1 to 5 candidates, this one to five numbers will be counted as the ballots of each perso. The vote counts will be like counters to be log points. If one of the numbers are tie for the highest score, it will trigger the tie true and vice versa, I printed both output, she the first output is Banaga won the votes, because in the unsorted array, using insertion array, the randomized number had become to hers, then on the secoon picture, Agustin and Yabut have become a tie.
Question:	Was your developed vote counting algorithm effective? Why or Why not?
	Yes it is, I have understand how the concept work. First, I need to have random element where it can have 100 elements. Second, use insertion sort, where I used this because its more effective, one by one put the balot and arrange them through, now sort the elements. Then, it's easier to see how many balots/elements that voted for each candidate, then I have seen how many and who one or got tied.

Pseudocode of Algorithm

```
MAIN
Begin main()
   // Define max size for array A[0...100]
   Constant max_size = 101
   Create integer array unsorted_arr[max_size]

   // Seed random numbers using current time
   Call srand(time(0))

   // Fill the array with random votes from 1 to 5
   For i = 0 To max_size - 1 Do
      unsorted_arr[i] = (rand() Mod 5) + 1
   End For

   // Print the unsorted array
   Print "Unsorted array: "
   Call displayArr(unsorted_arr, max_size)
   Print Newline

   // Run insertion sort algorithm
   Print "Sorted Array using Insertion Sort: "
   Call insertionSort_two(unsorted_arr, max_size)
   Call displayArr(unsorted_arr, max_size)
   Print Newline

   // Count votes and find who won
   Print "Election Results: "
   Call winnerVotes(unsorted_arr, max_size)
End main

Begin displayArr(arr, size)
   // Loop through array and print each element with a space
   For i = 0 To size - 1 Do
      Print arr[i] + " "
   End For
   Print Newline
End displayArr

Begin winnerVotes(arr, size)
   // Store names matching candidate numbers
   Create string array candidates[5] = [
      "Bo Dalton Capistrano",
      "Cornelius Raymon Agustin",
      "Deja Jayla Banaga",
      "Lalla Brielle Yabut",
```

```
    "Franklin Relano Castro"
]

// Initialize tally counters to 0
Create integer array vote_counts[5] = [0, 0, 0, 0, 0]

// Tally up the votes from the sorted array
For i = 0 To size - 1 Do
    Set vote = arr[i]
    If vote >= 1 And vote <= 5 Then
        // Subtract 1 because array indices start at 0
        vote_counts[vote - 1] = vote_counts[vote - 1] + 1
    End If
End For

// Track variables for finding the maximum votes
Set max_votes = -1
Set winner = 0
Set tie = False

// Loop through counts to display them and determine the winner
For i = 0 To 4 Do
    Print "Candidate " + (i + 1) + ":" + candidates[i] + " with " + vote_counts[i] + " votes"

    // Update winner if a higher vote count is found
    If vote_counts[i] > max_votes Then
        max_votes = vote_counts[i]
        winner = i
        tie = False
    Else If vote_counts[i] == max_votes Then
        tie = True
    End If
End For

Print Newline

// Print the final result text
If tie == True Then
    Print "The election is a TIE!"
Else
      Print "The winner of this election is " + candidates[winner] + "!!!! with the total votes of " +
max_votes + "!"
End If
End winnerVotes
```

HEADER

```
Begin insertionSort_two(arr, size)
   // Start from the second element
   For i = 1 To size - 1 Do
      Set temp = arr[i]
      Set j = i - 1

      // Move elements that are greater than temp one position ahead
      While j >= 0 And arr[j] > temp Do
         arr[j + 1] = arr[j]
         j = j - 1
      End While

      // Drop temp into its correct spot
      arr[j + 1] = temp
   END For
End insertionSort_two
```

Conclusion:

In this activity, I have learned more of the concepts behind each and one sorting algorithms, now I have the ability to decipher what insertion, selection and bubble sort differences is. Insertion is like the game of card, selection is where store then put, then bubble sort will be one by one checking. My conclusion in this activity is my realization where I still need to improve my skill in doing the code by scratch, by using psuedocode, it is much easier to visualize what I want to do, but the problem is where how to do it? Then my curiosity had lead where is it possible to link a pattern where a insertion sort and a linked list can be possible? It should be in different activity where it can be implemented, and Im a bit sure that it will be mind aching, but this activity is much more understandable, I just need to work on my coding skills.

3vb . ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of my idea of the code**

Examples: grammar correction, idea generation, formatting, summarization, code assistance,

explanation of concepts, etc.

Extent of Use: **The idea is original but it need to be double check for correct idea and thinking.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.